

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1

Выполнил:

Лошкарёв Д. В.

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

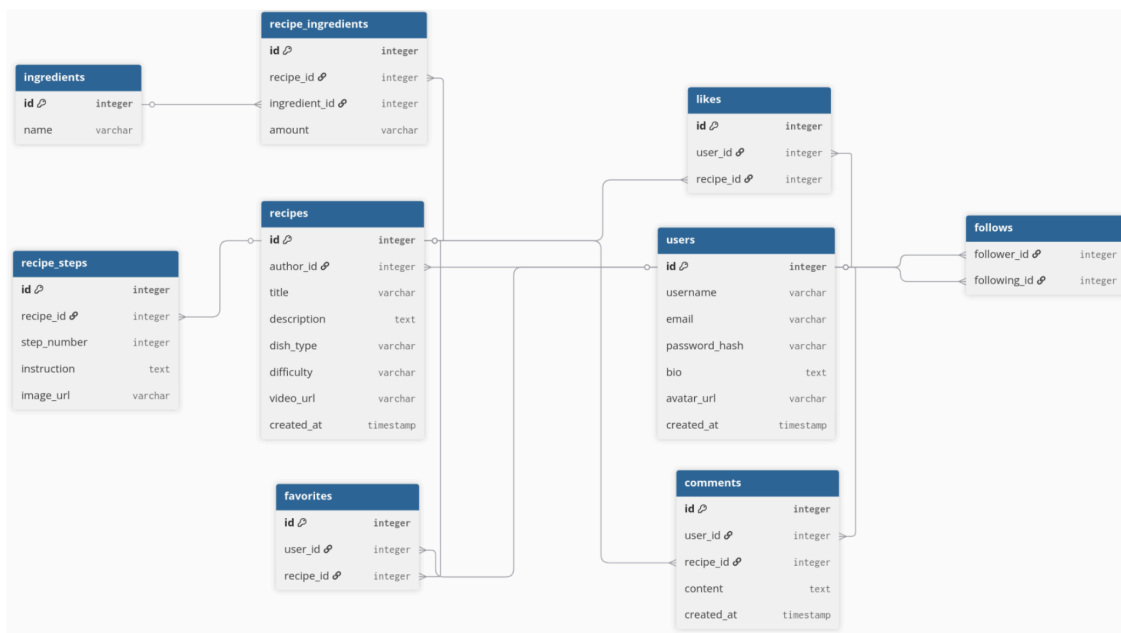
Задача

Основная задача этой работы заключалась в том, чтобы оживить проект и превратить сухие схемы из первых домашних заданий в реальное рабочее приложение. Мне нужно было создать полноценный бэкенд для сервиса обмена рецептами, используя Node.js и TypeScript. Важно было не просто написать код, а построить правильную архитектуру с моделями, контроллерами и защищенными маршрутами, а также подготовить базу данных, которая будет автоматически подстраиваться под описание в коде. В итоге я должен был получить готовое API, которое умеет регистрировать пользователей, искать рецепты по сложным фильтрам и управлять контентом.

Ход работы

Я начал работу с настройки окружения и сразу решил использовать Докер. Это позволило мне не засорять основную систему и быстро поднять связку из базы данных PostgreSQL, утилиты Adminer для просмотра таблиц и самого приложения. Весь проект я писал на TypeScript, потому что это стандарт для современной разработки, который помогает избегать глупых ошибок за счет типизации.

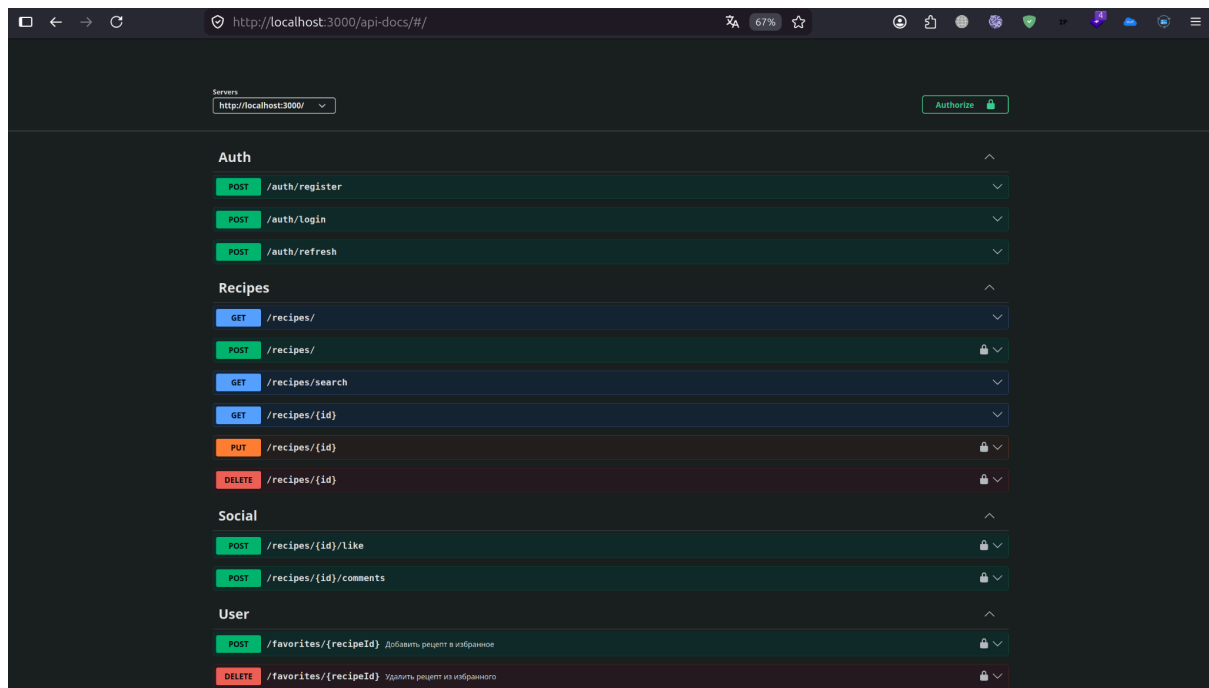
Первым делом я перенес свою схему базы данных в код. Я создал специальные классы, которые называются сущностями. В них я описал все 9 таблиц, которые планировал изначально: от профилей пользователей до сложных связей между рецептами, шагами приготовления и ингредиентами. Благодаря возможностям TypeORM, мне не пришлось писать SQL-запросы для создания таблиц, система сама поняла структуру кода и создала нужную архитектуру в базе, включая все связи и ключи.



Затем я занялся безопасностью и логикой входа. Я внедрил регистрацию и логин, где пароли обязательно хешируются перед сохранением, чтобы даже в случае утечки данных они были в безопасности. Для управления сессиями я выбрал JWT-токены. Чтобы защитить определенные действия, например, создание нового рецепта или удаление старого, я написал специальную проверку, которая смотрит на токен пользователя и разрешает или запрещает действие.

Когда базовая часть заработала, я перешел к самому интересному — логике рецептов. Я написал контроллеры, которые отвечают за добавление контента и поиск. Я реализовал продвинутый поиск, где можно отфильтровать блюда по сложности, типу или просто найти их по названию. Также я добавил проверку прав, чтобы никто не мог случайно или намеренно отредактировать чужой рецепт. Только автор имеет доступ к управлению своим контентом.

Для того чтобы API было удобно тестировать, я прикрутил автоматическую генерацию документации через Swagger. Я разделил все функции на понятные блоки: отдельно работа с пользователем, отдельно рецепты и социальные функции вроде лайков. В самом интерфейсе я добавил возможность авторизации, чтобы можно было проверять защищенные ручки прямо из браузера.



В конце я наполнил базу тестовыми данными через SQL-скрипт. Я создал пользователя test, добавил несколько популярных рецептов и проверил, как работают связи. Всё отработало как надо: рецепты привязываются к авторам, поиск находит нужные позиции, а система безопасности не пускает анонимных пользователей туда, где нужен токен.

Вывод

В результате выполнения лабораторной работы у меня получилось создать крепкий фундамент для будущего сервиса. Я на практике разобрался, как работают современные ORM-системы и как правильно организовывать структуру папок в проекте на Node.js. Самым полезным опытом стала работа с Docker и настройка связей между микросервисами внутри одной сети. Теперь у проекта есть надежное API с автодокументацией, которое полностью готово к интеграции с фронтендом или к дальнейшему расширению функционала.